recall

Purpose

The recall module reads externally provided time series and exposes them as spatial objects in the SWAT+ object graph. A recall object is a point source, an outlet, or a basin inlet whose flow and constituent loads are read from disk instead of simulated. Each recall has its own time step (subdaily, daily, monthly, or annual). The hydrograph delivered by a recall feeds the receiving object through `recall.con`, exactly the same way that an HRU or a channel feeds its downstream link.

Inputs come in two layers:

- A master database file (`recall.rec`) lists every recall object and points to the per-object side files for organic and mineral constituents and, optionally, for pesticides, pathogens, heavy metals, salts, and other constituents.
- Optional parallel database files (`salt_recall.rec`, `cs_recall.rec`) carry the time series of salt-ion and constituent loads when those modules are active.
- Optional spatial-element files (`rec_catunit.def`, `rec_reg.def`, `rec_catunit.ele`) group recall objects into regions used for output aggregation and soft calibration.

Source

- Master reader: `src/recall_read.f90` (`recalldb_read` and `recall_read`)
- Salt reader: `src/recall_read_salt.f90`
- Constituent reader: `src/recall_read_cs.f90`
- Element reader: `src/rec_read_elements.f90`
- Type definitions: `src/recall_module.f90` (`type recall_databases`, `type constituent_file_data`), `src/hydrograph_module.f90` (`type recall_hydrograph_inputs`), `src/constituent_mass_module.f90` (`type recall_salt_inputs`, `type recall_cs_inputs`, `type recall_pesticide_inputs`).

recall.rec (master database)

Lists every recall object and names the side files that hold its time-series data.

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one record per recall object. Two passes: the first finds the maximum object number, the second reads the records.

Each record is read into `recall_db(i)` (`type recall_databases`):

#	Field	Type	Description
1	id	int	Sequential id. Used as the array index.
2	name	char(13)	Recall object name. Referenced by `recall.con` and by the `obj_typ = rec` slots in other connectivity files.
3	org_min	record	Organic and mineral side file. Three subfields read together: `name` (filename), `units` (`mass` or `conc`), `tstep` (`sub`, `day`, `mo`, `yr`).
4	pest	record	Pesticide side file (same three subfields).
5	path	record	Pathogen side file.
6	hmet	record	Heavy-metal side file.
7	salt	record	Salt side file.
8	constit	record	Constituent (other) side file.

`type constituent_file_data` has three character fields read in order: `name(25)`, `units(13)`, `tstep(13)`. List-directed reads consume three tokens per record slot. After reading each row, the master reader calls `recall_read(i)` which opens the file in `recall_db(i)%org_min%name` and reads the time series.

Per-object time series

`recall_read` opens the organic-mineral side file and dispatches on `tstep`:

`tstep`	Storage	Records per year
`sub`	`recall(irec)%hyd_flo(step*366, nbyr)` plus `recall(irec)%hd(366, nbyr)`	Subdaily. Flow is converted from m^3/s to m^3 per subdaily step.
`day`	`recall(irec)%hd(366, nbyr)`	One row per Julian day.
`mo`	`recall(irec)%hd(12, nbyr)`	One row per month.
`yr`	`recall(irec)%hd(1, nbyr)`	One row per year.

Side-file format (`day`, `mo`, `yr`):

- Line 1: title.
- Line 2: `nbyr` (number of years).
- Line 3: header (skipped).
- Lines 4+: per time step, `jday`, `mo`, `day_mo`, `iyr`, `ob_typ`, `ob_name`, then a 22-field `hyd_output` record (flow plus organic, mineral, and physical constituents).

Each `hyd_output` row supplies, in order: `flo`, `sed`, `orgn`, `sedp`, `no3`, `solp`, `chla`, `nh3`, `no2`, `cbod`, `dox`, `san`, `sil`, `cla`, `sag`, `lag`, `grv`, `temp`. Flow is in m^3/s. Sediment is in metric tons. Nutrients and other constituents are in kg. See `type hyd_output` for the full column list.

The reader fast-forwards the side file to the row whose `iyr` matches `time%yrc` (simulation start year). Rows before the simulation start are skipped; rows after `time%yrc_end` end the read. Each row sets `recall(irec)%start_yr` to the first matching year and `recall(irec)%end_yr` to the last row read.

pest.com (recall pesticide side database)

When `pest.com` exists in the working directory, `recall_read` reads it as a parallel pesticide database. It is structured like `recall.rec`: a master file naming per-pesticide files. Each per-pesticide file has its own `nbyr` and time step (1=day, 2=mo, 3=yr) and fills `rec_pest(i)%hd_pest(istep, nbyr)`. The exact column count of the per-pesticide payload depends on `db_mx%pestcom` (the number of pesticides in the constituent database).

salt_recall.rec (salt side database)

Mirror of `recall.rec` for salt loads, read when the salt module is active.

- Line 1: title.
- Line 2: header.
- Lines 3+: one record per recall object. Fields: `id`, `name`, `typ` (1=day, 2=mo, 3=yr, 4=exco crosswalk), `filename`.

For `typ` in `1..3`, the side file is opened with:

- Line 1: title. If the literal token is `Incoming` the source is flagged as originating from outside the watershed (`pts_type = 2`); otherwise `pts_type = 1` (within the watershed).
- Line 2: `nbyr`.
- Line 3: header.
- Lines 4+: `jday`, `mo`, `day_mo`, `iyr`, `ob_typ`, `ob_name`, then `cs_db%num_salts` real values (one per salt ion). The order of ions follows `constituents.cs`.

Each side file fills `rec_salt(i)%hd_salt(istep, nbyr)%salt(1..num_salts)`. For `typ = 4`, the salt record is crosswalked to an `exco` entry; no side file is opened.

cs_recall.rec (constituent side database)

Same structure as `salt_recall.rec`, but for the other-constituent module. Fields per record: `id`, `name`, `typ`, `filename`. Side files have the same five-line header pattern; payload rows carry `cs_db%num_cs` real values that fill `rec_cs(i)%hd_cs(istep, nbyr)%cs(1..num_cs)`. The `Incoming` flag in the title is honoured identically.

Element and region files

Three files in `file.cio`'s `regions` slot group recall objects for output aggregation and soft calibration. Names default to `rec_catunit.def`, `rec_reg.def`, and `rec_catunit.ele` (set by `in_regs%def_psc`, `in_regs%def_psc_reg`, and `in_regs%ele_psc`). All three are optional.

rec_catunit.def (output regions)

Defines the regions used to print recall output. Format:

- Line 1: title.
- Line 2: number of regions, `mreg`.
- Line 3: header.
- Lines 4+: per region: `id`, `name`, `area_ha`, `nspu`, then `nspu` element ids.

When `nspu > 0`, the listed element ids are passed to `define_unit_elements` and stored in `pcu_out(i)%num`. When `nspu = 0`, the region defaults to every recall in the basin (`sp_ob%recall` elements).

rec_reg.def (calibration regions)

Same format as `rec_catunit.def`. Populates `pcu_reg(i)` instead of `pcu_out(i)`. Used for soft-calibration grouping and per-type output.

rec_catunit.ele (element table)

Lists every element that can appear in `rec_catunit.def` or `rec_reg.def`. Format:

- Line 1: title.
- Line 2: header.
- Lines 3+: per element: `id`, `name`, `obtyp`, `obtypno`, `bsn_frac`, `ru_frac`, `reg_frac`.

`obtyp` is the object-type tag (typically `rec`) and `obtypno` is the recall object number. The three fractions are the share of the element in the basin, in its routing unit, and in its region.

Example

`refdata/Osu_1hru/recall.rec`:

```
recall.rec
NUM     NAME    TYPE     FNAME    OBJ_TYP     OBJ_NAME       flo      sed     orgn     sedp      no3     solp
   1    PSTYR    3      RECYR.        hru         hru1   12120.5        0        0        0        0        0
   1      1          1    2011        hru         hru2    9000.7        0        0        0        0        0
```

This dataset stores one recall object (`PSTYR`) whose annual hydrograph lives in a side file. The trailing 22-column row is the actual `hyd_output` payload that the side file would contain. In a clean layout, the side file (`RECYR.` or similar) is its own file: a title line, a year count, a header, then one or more annual rows.

Related files

- recall.con. Spatial-connectivity entries for each recall object. Links the recall to its receiving spatial object.
- file.cio. Names the master `recall.rec` (`in_rec%recall_rec`) and the element files (`in_regs%def_psc`, `in_regs%def_psc_reg`, `in_regs%ele_psc`).
- salt module, cs module. Document the salt and other-constituent modules whose recall side databases parallel `recall.rec`.